Lecture 2: Introduction to R

We will look at:

- What is R?
- Basic R syntax

MASSEY UNIVERSITY
TE KUNENGA KI PŪREHUROA
UNIVERSITY OF NEW ZEALAND

What is R?

- R is a statistical software system.
- R is a programming language that has many “inbuilt” statistical commands (e.g. to fit a linear regression).
- R started life as a quasi-clone of commercial package S-Plus, but development of R and S-Plus now slowly diverging.
- Advantages over other statistics packages include flexibility, power, and quality of graphical display.
- R is open-source software, and part of GNU project. It can be downloaded from `http://cran.r-project.org/` and used for free.
- There are versions of R for all common operating systems – Windows, Linux, and MacOS.

A Little R History




Ross Ihaka (left) and Robert Gentleman (right)

- R is kiwi!
- R originally developed by Ross Ihaka and Robert Gentleman in the Department of Statistics, University of Auckland.
- First (test) version of R released in public domain in 1995.
- R Development Core Team took over supervision of R in 1997.
- This Team includes over 20 statisticians and computer scientists worldwide.

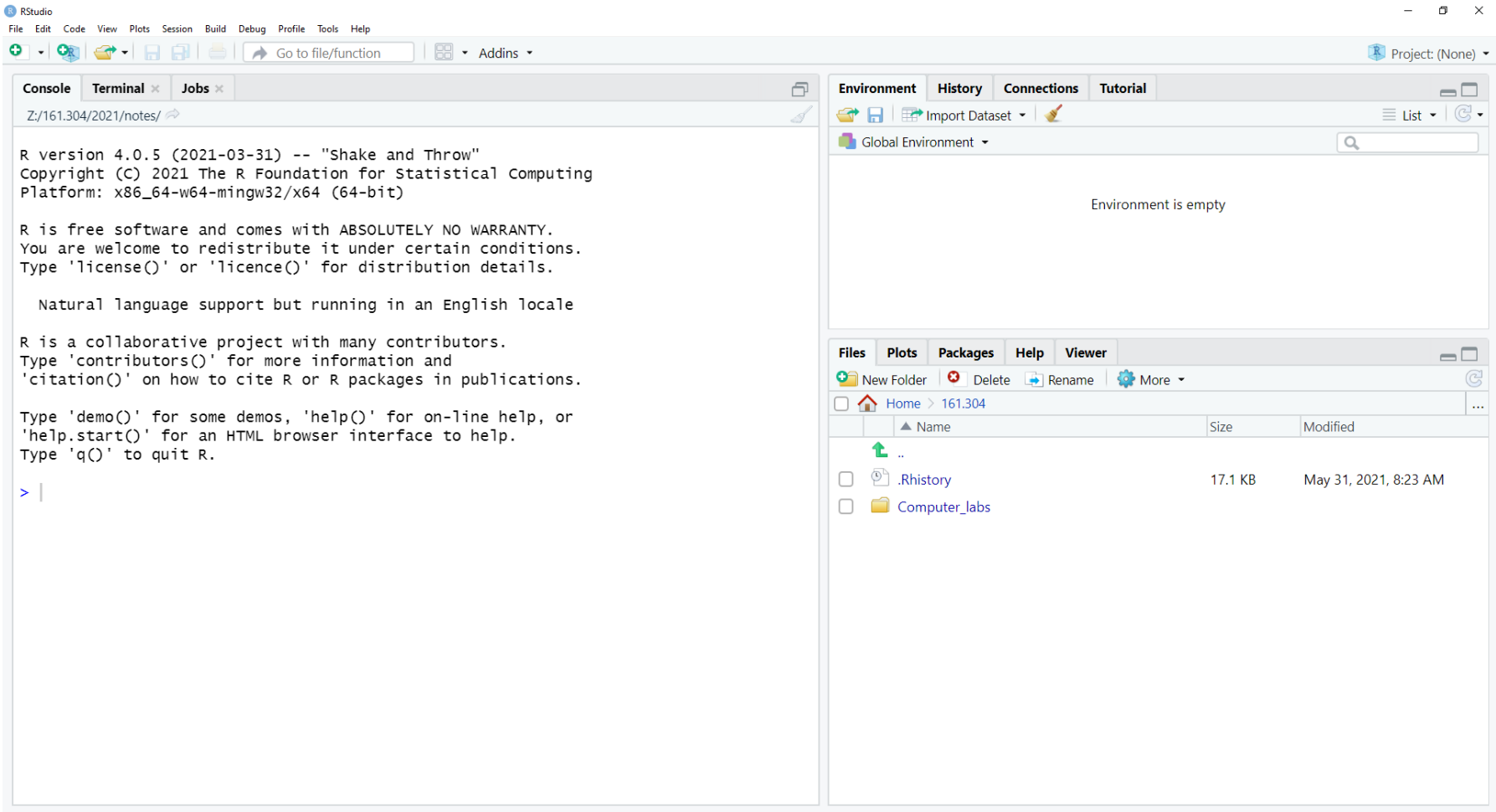
MASSEY UNIVERSITY
TE KUNENGA KI PŪREHUROA
UNIVERSITY OF NEW ZEALAND

RStudio

- RStudio is a IDE (integrated development environment) for R
- Provides a command line interface and a built-in editor for R scripts
- Available for all platforms
- Provides advanced functionalities like version control and project management



RStudio


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Project: (None)
Console
Terminal
Jobs
Z:/161.304/2021/notes/
R version 4.0.5 (2021-03-31) -- "Shake and Throw"
Copyright (C) 2021 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>
Environment
History
Connections
Tutorial
Import Dataset
List
Global Environment
Environment is empty
Files
Plots
Packages
Help
Viewer
New Folder
Delete
Rename
More
Home
161.304
Name
Size
Modified
..
.Rhistory
17.1 KB
May 31, 2021, 8:23 AM
Computer_labs

Setting the Working Directory

- When you quit RStudio you can save your work.
- If you choose to save work, R will retain:
  - The *R workspace* that you have created;
  - An *R history file*, listing of all the R commands that you issued,
- These files are saved in your *working directory*.
- By default the working directory is something like `C:/Documents and Settings/`*Your User Name*`/My Documents`.
- The working directory can be changed using the `Session` menu, by selecting `Set working directory > Choose directory`.
- Saved R workspace then represented by icon in working directory – double click this to restart R with that workspace.

A First Dip into R

Expressions and Assignments

Elementary commands are either *expressions* or *assignments*.

- An *expression* simply displays result of a calculation; not retained in the computer's memory.
- An *assignment* passes the result of a calculation to a variable name (or 'object') which is stored; the result is not displayed.

Expressions and Assignments

Examples

R Snippet (Expression)

```
> 3+4
[1] 7
```

- The symbol > is the command prompt.
- Don't worry too much at this stage about the `[1]`.

R Snippet (Assignment)

```
> x <- 3 + 4
> x
[1] 7
```

- <− to indicate assignment from right (3 + 4) to left (x)

R Objects

- All assigned variables (or any other R *objects*) are stored until overwritten or explicitly removed (deleted) by the command `rm`.
- To list stored objects type `ls()` or `objects()`.

R Snippet

```
> x <- 8
> y <- 3.1415
> ls()
[1] "x" "y"
> rm(x)
> objects()
[1] "y"
```

R Syntax

- R commands, e.g. `ls`, `rm`, are followed by parentheses which may contain additional information for the function.
- Writing a command name without parentheses returns the R source code for the function (not always that simple).

R Snippet

```
> rm(y)
> rm
function (..., list = character(0), pos = -1,.....
    inherits = FALSE)
{
    dots <- match.call(expand.dots = FALSE)...
    if (length(dots) && !all(sapply(dots, function(x)
        is.character(x))))
.....
```

Vectors in R

- The command `c()` (for *concatenate*) creates vectors.

R Snippet

```
> x <- c(2.3,1.2,2.4)
> x
[1] 2.3 1.2 2.4
> c(x,9.0,x)
[1] 2.3 1.2 2.4 9.0 2.3 1.2 2.4
```

MASSEY UNIVERSITY
TE KUNENGA KI PŪREHUROA
UNIVERSITY OF NEW ZEALAND

Vectors in R

Regular Sequences

- The expression `1:n` denotes the **sequence** $1, 2, \ldots, n$.
- The expression `seq(i,j,by=k)` is a sequence from i to j in steps of k.
- Explore the `each` option in `seq()`

R Snippet

```
> 1:5
[1] 1 2 3 4 5
> y <- seq(3,10,by=2)
> y
[1] 3 5 7 9
```

Vector Arithmetic in R

- R uses `+`, `-`, `*` and `/` for the basic arithmetic operations, and `^` for exponentiation (raising to a power).
- Vector operations are done element by element...with recycling of short vectors if required (make sure you know what you are doing!).

R Snippet

```
> x <- c(2,3)
> y <- c(1,4,5,6)
> 2*x
[1] 4 6
> 2 + x
[1] 4 5
> y^2
[1] 1 16 25 36
> x + y
[1] 3 7 7 9
```

Types of vector

- All the vectors we have seen so far have been *numeric*.
- R also understands vectors of:
  - characters
  - logical values `TRUE` and `FALSE`; abbreviations `T`, `F` OK.
  - factors (i.e. categorical variables)

R Snippet

```
> mywords <- c("This","is","a","character")
> mywords
[1] "This"      "is"        "a"         "character"
> c(F,T,F,F)
[1] FALSE TRUE FALSE FALSE
> factor(c("Low","Low","Medium","High","High"))
[1] Low    Low    Medium High   High
Levels: High Low Medium
```

Logical Comparisons

- Numerical vectors can be compared by inequalities.
- == denotes equality.
- != denotes 'not equal'.
- >, >= etc. for inequalities.

R Snippet

```
> (1:5) == (5:1)
[1] FALSE FALSE  TRUE FALSE FALSE
> (1:5) > (5:1)
[1] FALSE FALSE FALSE  TRUE  TRUE
```

Indexing Vectors

- To index components of a vector `x`, use the form `x[...]`.
- The square brackets can contain:
  - numeric vector specifying elements;
  - logical vector: only `TRUE` elements required.

R Snippet

```
> x <- c(1.1,3.2,4.3,7.4)
> x[c(2,4)]
[1] 3.2 7.4
> x[-2]
[1] 1.1 4.3 7.4
> x[x > 3.5]
[1] 4.3 7.4
> which(x > 3.5)
[1] 3 4
```

Data Frames

- A data frame is a collection of column vectors each of the same length.
- The vectors may be numeric, factor, or whatever.
- Each particular column and row of a data frame is given a name which can be chosen by the user, or assigned a default by R.

R Snippet

```
> employee <- c("Dilbert","Wally","Catbert","TheBoss")
> job<-factor(c("Engineer","Engineer","Manager","Manager"))
> x <- c(8,1,NA,-2)
> dilbert <- data.frame(employee, job, competence=x)
> dilbert
  employee      job  competence
1  Dilbert  Engineer           8
2    Wally  Engineer           1
3  Catbert   Manager          NA
4  TheBoss   Manager          -2
```

Data Frames

Attaching and Detaching

- To access variables (columns) of a data frame:
  - First `attach` data frame; or
  - Use `data.frame$variable` syntax (safer).

R Snippet

```
> rm(employee,job,x)
> dilbert$competence
[1]  8  1  NA  -2
> job
Error: Object "job" not found
> attach(dilbert)
> job
[1] Engineer  Engineer  Manager  Manager
Levels:  Engineer  Manager
> detach(dilbert)
```

Importing Data

- The `scan` command reads in text from a file as a single variable.
- The `read.table` command is more flexible, importing data in tabular form and storing the result as a data frame.

R Snippet

```
> ibm <- scan(file="ibm.txt")
Read 250 items
> ibm
  [1] 64.37 62.50 63.50 63.37 63.12 67.37 65.37 .....
.....
[239] 68.12 66.50 67.87 68.00 68.37 67.50 66.37 .....
> life <- read.table(file="life.txt", header=T)
> life
        Country LifeExp People.per.TV People.per.Dr  .....
1     Argentina    70.5            4.0           370  .....
.....
40        Zaire    54.0             NA         23193  .....
```

Editing Data

- Can edit by reassigning elements of a vector or data frame.
- For data frame or matrix, `A[i,j]` is the i,jth element (row i, column j) of the data frame `A`.
- Alternatively, `edit` provides basic spreadsheet function.

R Snippet

```
> ibm[5]
[1] 63.12
> ibm[5] <- 65.12
> life[2,]
     Country LifeExp People.per.TV People.per.Dr .....
2 Bangladesh    53.5           315          6166 .....
> life[2,4]
[1] 6166
> life[2,4] <- 7166
```

R Packages

- R objects (functions, data etc.) are organized in libraries/packages.
- Some are loaded by default when R starts each time.
  - E.g. function `ls` is part of the `base` package that is automatically loaded,
- Some packages need loading, using the `library` command.

R Snippet

```
> mvrnorm(1,mu=0,Sigma=1)
Error: could not find function "mvrnorm"
> library(MASS)
> mvrnorm(1,mu=0,Sigma=1)
[1] 1.091034
```

Some R Functions to Get You Started

- `help` accesses R's help system; e.g. `help(ls)`.
- `mean`, `sd`, `min`, `max` and `range` give mean, standard deviation, minimum, maximum and range respectively for a vector argument.
- `var` returns variance of a vector argument, or the covariance (dispersion) matrix for a matrix argument.
- `summary` returns summary information dependent on argument type.
- `plot` produces a plot on the current graphics tool. The type of plot depends on the type of argument. Simplest is `plot(x,y)` which produces a scatter-plot of vectors `x` and `y`.

Practice is the key when learning R.

Task 1: Your First R Exercise

1. Use R to calculate (i) $3456 - 789$; (ii) 23×34; (iii) 13^3.
2. Write (efficient) code to create the following sequences:
   (a) $2, 4, 6, \ldots, 100$ (even numbers up to 100)
   (b) $1, 2, 3, 4, 5, 4, 3, 2, 1$
3. Use the command `y <- rnorm(100)` to store 100 simulated standard normal random variables in the vector `y`.
   (a) Find the mean and standard deviation of `y`.
   (b) Find the largest simulated value. Which number simulation is this (e.g. 54th, 23rd?)

The `rethinking` package

- Once you have R and then Rstudio installed
- Follow the instructions at `https://github.com/rmcelreath/rethinking` to install in this order (i) `rstan`, (ii) `cmdstanr` and (iii) `rethinking`.